

Hadoop与Spark

Spark MLLib



Spark MLlib

◆ Spark MLlib简介

- ◆ 传统的机器学习算法，由于单机存储的限制，只能在少量数据上运行，迫使我们有时候首先对大数据进行抽样，才能进行后续处理。在数据抽样上进行机器学习，训练得到的模型不是很准确。随着Hadoop(MapReduce)、Spark等大数据平台的流行，在全量数据上进行学习已经成为可能。
- ◆ 在Spark平台上运行机器学习算法，相对于MapReduce计算平台，区别如下：
 - ◆ (1) 机器学习算法的计算，一般需要多趟迭代过程，才能获得足够小的误差，达到收敛然后停止。如果使用Hadoop的MapReduce计算框架，每次迭代都要读/写磁盘，I/O消耗很大。此外Map/Reduce任务的启动也消耗时间。
 - ◆ (2) Spark是基于内存计算模型，多次迭代直接在内存中完成，只有在必要时才会写入磁盘、或者通过网络交换数据。Spark是机器学习的理想的平台。
- ◆ Mllib是Spark平台的机器学习组件，直接操作RDD数据集，天生就可以与Spark SQL、GraphX、Spark Streaming等组件无缝集成

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ MLlib目前支持4种常见的机器学习问题: 分类、回归、聚类和协同过滤

MLlib 支持的机器学习算法

↵	离散数据↵	连续数据↵	↵
有监督学习↵	Classification、LogisticRegression、SVM、DecisionTree 、 RandomForest 、 NaiveBayes、MultilayerPerceptron、GBT、OneVsRest...↵	LinearRegression 、 Regression DecisionTree、 RandomForest 、 GBT、 AFTSurvivalRegression 、 IsotonicRegression...↵	↵
无监督学习↵	Kmeans Clustering、 BisectingKMeans 、 GaussianMixture 、 PowerIterationClustering、 Latent Dirichlet Allocation (LDA)...↵	Dimensionality Reduction 、 Matrix Factorization、 PCA、 SVD、 Alternating Least Squares (ALS)、 Weighted Least Squares (WLS)...↵	↵

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ Spark机器学习库从1.2版本以后被分为两个包:
 - ◆ (1) `spark.mllib`, 包含基于RDD的原始算法API
 - ◆ (2) `spark.ml`, 则提供了基于DataFrames高层次的API, 可以用来构建机器学习工作流(Pipeline), 有利于在整个工作流中集成特征抽取、转换功能; 此外, 在DataFrame基础上, Spark提供了跨语言的统一的API, 有利于开发人员掌握Spark编程
 - ◆ Spark官方推荐使用ml, 因为ml功能更全面更灵活, 未来会主要支持ml (在Spark3.0中, mllib将被彻底废弃)

pause



Spark MLlib

- ◆ Spark MLlib简介
 - ◆ Mllib提供了大量的分类算法，包括Logistic Regression、Decision Tree、Random Forest、Gradient-Boosted Tree、Multilayer Perceptron Classifier、Support Vector Machine、One-vs-Rest Classifier、Naive Bayes等
 - ◆ 这些算法的实例都可以在“/opt/linuxsir/spark/examples”目录下找到，数据文件则在“/opt/linuxsir/spark/data”目录下

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 1, 分类实例
 - ◆ 基于决策树分类的示例代码

```
#首先导入必要的类。
from pyspark.ml import Pipeline
from pyspark.ml.classification import DecisionTreeClassifier
from pyspark.ml.feature import StringIndexer, VectorIndexer
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

#创建SparkSession。
spark = SparkSession\
    .builder\
    .appName("DecisionTreeClassificationExample")\
    .getOrCreate()
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 1, 分类实例
 - ◆ 基于决策树分类的示例代码

```
#装载数据文件到内存，形成DataFrame。  
# Load the data stored in LIBSVM format as a DataFrame.  
data =  
spark.read.format("libsvm").load("file:///opt/linuxsir/spark/data/mlli  
b/sample_libsvm_data.txt")
```

- ◆ 数据文件具有特定的格式，每一行具有如下的形式。
- ◆ 0 128:51 129:159 130:253 131:159 132:50 155:48 156:238 157:252 158:252 159:252……
- ◆ 数据格式为“标签 特征1 特征2 特征3 …”。对于二值分类，标签有两个取值，即0和1。由于每个样本为一个稀疏的高维向量，所以对于每个特征采用“特征编号: 特征值”的编码方式，比如“128:51”，表示第128号特征取值为51，没有列出来的其它特征取缺省值(0)

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 1, 分类实例
 - ◆ 基于决策树分类的示例代码

```
#把每行数据的字符型标签(label)映射到一个整数(indexedLabel)。
```

```
# Index labels, adding metadata to the label column.
```

```
# Fit on whole dataset to include all labels in index.
```

```
labelIndexer = StringIndexer(inputCol="label",  
outputCol="indexedLabel").fit(data)
```

```
#对不同值的数量<=4的特征，认为是离散特征，每个特征的取值映射到一个整数；对于其它特征，则认为是连续型特征。
```

```
# Automatically identify categorical features, and index them.
```

```
# We specify maxCategories so features with > 4 distinct values are treated as continuous.
```

```
featureIndexer =\  
    VectorIndexer(inputCol="features", outputCol="indexedFeatures",  
maxCategories=4).fit(data)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 1, 分类实例
 - ◆ 基于决策树分类的示例代码

```
#划分训练数据集和测试数据集。
# Split the data into training and test sets (30% held out for testing)
(trainingData, testData) = data.randomSplit([0.7, 0.3])

#创建DecisionTreeClassifier类的一个对象，即决策树模型。
# Train a DecisionTree model.
dt = DecisionTreeClassifier(labelCol="indexedLabel",
featuresCol="indexedFeatures")

#建立流水线，包含数据预处理和决策树模型等不同构件，这些构件也可以看作是
# 数据处理的不同阶段。
# Chain indexers and tree in a Pipeline
pipeline = Pipeline(stages=[labelIndexer, featureIndexer, dt])

#把训练数据馈入流水线，训练机器学习模型。
# Train model. This also runs the indexers.
model = pipeline.fit(trainingData)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 1, 分类实例
 - ◆ 基于决策树分类的示例代码

```
#利用已经训练好的模型，进行预测。
```

```
# Make predictions.
```

```
predictions = model.transform(testData)
```

```
#显示模型的预测结果，最多显示5条。
```

```
# Select example rows to display.
```

```
predictions.select("prediction", "indexedLabel", "features").show(5)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 1, 分类实例
 - ◆ 基于决策树分类的示例代码

```
#对模型进行评价，评价指标是模型的正确率(Accuracy)。  
#在下文中，有时候模型指的是机器学习模型，有时候模型指的是完整的流水线，  
#请读者加以区分。  
# Select (prediction, true label) and compute test error  
evaluator = MulticlassClassificationEvaluator(  
    labelCol="indexedLabel", predictionCol="prediction",  
    metricName="accuracy")  
accuracy = evaluator.evaluate(predictions)  
print("Test Error = %g " % (1.0 - accuracy))
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 1, 分类实例
 - ◆ 基于决策树分类的示例代码

```
#显示训练后的决策树的信息。  
treeModel = model.stages[2]  
# summary only  
print(treeModel)  
  
#停止SparkSession。  
spark.stop()
```

pause



Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 2, 文本分类
 - ◆ 参考<https://github.com/ayoyu/Spark ML Classification of texts>

首先把数据文件上传到HDFS。

```
cd /opt/linuxsir/hadoop/bin
```

```
./hdfs dfs -copyFromLocal /opt/linuxsir/20ng-train-all-terms.txt /input
```

```
./hdfs dfs -copyFromLocal /opt/linuxsir/20ng-test-all-terms.txt /input
```

```
./hdfs dfs -ls /input
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 2, 文本分类
 - ◆ 参考<https://github.com/ayoyu/Spark ML Classification of texts>

#首先, 导入必要的类。

```
from pyspark import SparkContext
from pyspark.sql import Row
from pyspark.sql import SparkSession
from pyspark.ml.feature import CountVectorizer
from pyspark.ml.feature import StringIndexer
from pyspark.ml.classification import NaiveBayes
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.ml import Pipeline
```

#创建SparkSession。

```
sc = SparkContext()
spark = SparkSession.builder.getOrCreate()
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 2, 文本分类
 - ◆ 参考<https://github.com/ayoyu/Spark ML Classification of texts>

```
#定义数据装载函数。
def load_data(path):
    rdd = sc.textFile(path).\
        map(lambda line:line.split()).map(lambda
word:Row(label=word[0],words=word[1:]))
    return spark.createDataFrame(rdd)

#装载训练集和测试集。
train_data = load_data("/input/20ng-train-all-terms.txt")
test_data = load_data("/input/20ng-test-all-terms.txt")
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 2, 文本分类
 - ◆ 参考<https://github.com/ayoyu/Spark ML Classification of texts>

```
#创建CountVectorizer、StringIndexer、以及NaiveBayes分类器。关于StringIndexer，请参考前一个实例的说明。
vectorizer = CountVectorizer(inputCol = 'words',outputCol='bag_of_words')
label_indexer = StringIndexer(inputCol = 'label', outputCol = 'label_index')
classifier_naive = NaiveBayes(labelCol = 'label_index', \
    featuresCol = 'bag_of_words', predictionCol = 'label_pred')

#在CountVectorizer、StringIndexer、以及NaiveBayes分类器基础上，构建Pipeline。并且用训练数据进行训练。
pipeline = Pipeline(stages = [vectorizer, label_indexer, classifier_naive])
pipeline_model = pipeline.fit(train_data)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 2, 文本分类
 - ◆ 参考<https://github.com/ayoyu/Spark ML Classification of texts>

```
#利用测试集进行验证，计算分类器的正确率。  
test_pred = pipeline_model.transform(test_data)  
evaluator = MulticlassClassificationEvaluator(labelCol = 'label_index', \  
    predictionCol = 'label_pred', metricName = 'accuracy')  
  
accuracy = evaluator.evaluate(test_pred)  
print('NaiveBayes model accuracy_score = {:.2f}'.format(accuracy))  
#该NaiveBayes分类器的正确率为0.80，还可以
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 2, 文本分类
 - ◆ 参考<https://github.com/ayoyu/Spark ML Classification of texts>

#提取并显示测试集的前30行预测结果。

```
test_pred.select("label", "words", "label_index", "label_pred").show(30)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 2, 文本分类
 - ◆ 参考<https://github.com/ayoyu/Spark ML Classification of texts>

#对单独一行数据进行预测。

```
one_row_all = ['alt.atheism re societally acceptable behavior in qvh tinnsg citation ksu ksu edu yohan citation  
ksu ksu edu jonathan w newton writes in article c qgm dl news cso uiuc edu cobb alexia lis uiuc edu mike cobb  
writes merely a question for the basis of morality moral ethical behavior societally acceptable behavior i  
disagree with these what society thinks should be irrelevant what the individual decides is all that is important  
this doesn t seem right if i want to kill you i can because that is what i decide who is society i think this is fairly  
obvious not really if whatever a particular society mandates as ok is ok there are always some in the society  
who disagree with the mandates so which societal mandates make the standard for morality how do they  
define what is acceptable generally by what they feel is right which is the most idiotic policy i can think of so  
what should be the basis unfortunately i have to admit to being tied at least loosely to the feeling in that i think  
we intuitively know some things to be wrong awfully hard to defend though how do we keep from a whatever is  
legal is what is moral position by thinking for ourselves i might agree here just because certain actions are  
legal does not make them moral mac michael a cobb and i won t raise taxes on the middle university of illinois  
class to pay for my programs champaign urbana bill clinton rd debate cobb alexia lis uiuc edu with new taxes  
and spending cuts we ll still have billion dollar deficits michael a cobb and i won t raise taxes on the middle  
university of illinois class to pay for my programs champaign urbana bill clinton rd debate cobb alexia lis uiuc  
edu nobody can explain everything to anybody g k chesterton']
```

```
one_row_all_rdd = sc.parallelize(one_row_all)
```

```
one_row_rdd = one_row_all_rdd.map(lambda line:line.split()).map(lambda  
word:Row(label=word[0],words=word[1:]))
```

```
one_row_df = spark.createDataFrame(one_row_rdd)
```

```
one_row_pred = pipeline_model.transform(one_row_df)
```

```
one_row_pred.select("label", "words", "label_index", "label_pred").show(30)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 2, 文本分类
 - ◆ 参考<https://github.com/ayoyu/Spark ML Classification of texts>

```
#最后停止SparkSession。  
spark.stop()
```

pause



Spark MLlib

- ◆ Spark MLlib简介
 - ◆ MLlib提供K-means、Latent Dirichlet allocation (LDA)、Bisecting k-means、Gaussian Mixture Model (GMM)等聚类算法

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例3: 聚类实例

```
#首先, 导入必要的类。
from pyspark.ml.clustering import KMeans
from pyspark.sql import SparkSession

#创建SparkSession。
spark = SparkSession\
    .builder\
    .appName("KMeansExample")\
    .getOrCreate()
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例3: 聚类实例

```
#接着，装载数据。  
# Loads data.  
dataset =  
spark.read.format("libsvm").load("file:///opt/linuxsir/spark/data/mllib  
/sample_kmeans_data.txt")
```

数据文件具有特定的格式，文件内容如下。

```
0 1:0.0 2:0.0 3:0.0  
1 1:0.1 2:0.1 3:0.1  
2 1:0.2 2:0.2 3:0.2  
3 1:9.0 2:9.0 3:9.0  
4 1:9.1 2:9.1 3:9.1  
5 1:9.2 2:9.2 3:9.2
```

每一行的第一个数字为序号，接着给出每个特征的取值，每个特征采用“特征编号：特征值”的编码方式

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例3: 聚类实例

```
#创建K-Means模型，并进行训练。
```

```
# Trains a k-means model.
```

```
kmeans = KMeans().setK(2).setSeed(1)
```

```
model = kmeans.fit(dataset)
```

```
#利用训练好的模型，进行预测，即预测每个数据点的Cluster。
```

```
# Make predictions
```

```
predictions = model.transform(dataset)
```

```
predictions.select("prediction", "label", "features").show(5)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例3: 聚类实例

```
#对K-Means模型进行评价。  
# Evaluate clustering by computing Within Set Sum of Squared Errors.  
wssse = model.computeCost(dataset)  
print("Within Set Sum of Squared Errors = " + str(wssse))
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例3: 聚类实例

```
#显示类簇的中心。  
# Shows the result.  
centers = model.clusterCenters()  
print("Cluster Centers: ")  
for center in centers:  
    print(center)  
  
#停止SparkSession。  
spark.stop()
```

pause



Spark MLlib

- ◆ Spark MLlib简介
 - ◆ MLlib提供了Linear Regression、Generalized Linear Regression、Decision Tree Regression、Random Forest Regression、Gradient-Boosted Tree Regression、Survival Regression、Isotonic Regression等回归算法

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例4: 线性回归

```
#首先, 导入必要的类。
from pyspark.ml.regression import LinearRegression
from pyspark.sql import SparkSession

#创建SparkSession。
spark = SparkSession\
    .builder\
    .appName("LinearRegressionWithElasticNet")\
    .getOrCreate()
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例4: 线性回归

```
#装载数据，准备训练机器学习模型。  
# Load training data  
training = spark.read.format("libsvm")\  
    .load("file:///opt/linuxsir/spark/data/mllib/sample_linear_regressi  
on_data.txt")
```

数据文件具有特定的格式，每一行的格式如下。

```
-9.490009878824548 1:0.4551273600657362  
2:0.36644694351969087 3:-0.38256108933468047 4:-  
0.4458430198517267 5:0.33109790358914726  
6:0.8067445293443565 7:-0.2624341731773887 8:-  
0.44850386111659524 9:-0.07269284838169332  
10:0.5658035575800715
```

第一个数值表示因变量，接着给出每个特征的取值，每个特征采用“特征编号：特征值”的编码方式。

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例4: 线性回归

创建线性回归模型，并进行训练。

```
lr = LinearRegression(maxIter=10, regParam=0.3,  
elasticNetParam=0.8)
```

```
# Fit the model
```

```
lrModel = lr.fit(training)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例4: 线性回归

```
#显示线性回归模型的(各个变量的)系数和截距。  
# Print the coefficients and intercept for linear regression  
print("Coefficients: %s" % str(lrModel.coefficients))  
print("Intercept: %s" % str(lrModel.intercept))  
  
#显示迭代次数和目标函数变化情况(一般为RMSE)。  
# Summarize the model over the training set and print out some  
metrics  
trainingSummary = lrModel.summary  
print("numIterations: %d" % trainingSummary.totalIterations)  
print("objectiveHistory: %s" % str(trainingSummary.objectiveHistory))
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例4: 线性回归

```
#显示模型的残差、RMSE指标、和R2指标。  
trainingSummary.residuals.show()  
print("RMSE: %f" % trainingSummary.rootMeanSquaredError)  
print("r2: %f" % trainingSummary.r2)  
  
#停止SparkSession。  
spark.stop()
```

pause



Spark MLlib

- ◆ Spark MLlib简介
 - ◆ MLlib提供了Collaborative Filtering(协同过滤推荐)算法, 用于推荐
 - ◆ 实例5: 协同过滤推荐

```
#首先, 导入必要的类。
from pyspark.sql import SparkSession
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.recommendation import ALS
from pyspark.sql import Row

#创建SparkSession。
spark = SparkSession\
    .builder\
    .appName("ALSExample")\
    .getOrCreate()
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例5: 协同过滤推荐
 - ◆ 读取数据文件。

文件采用特殊的格式，前五行内容如下。每行由用“::”分割开的三个数值组成，这三个数值表示用户id、电影id、以及评分。

0::2::3

0::3::1

0::5::2

0::9::4

0::11::1

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例5: 协同过滤推荐
 - ◆ 数据文件处理

```
lines =  
spark.read.text("file:///opt/linuxsir/spark/data/mllib/als/sample_movielens_ratings.txt").rdd  
parts = lines.map(lambda row: row.value.split("::"))  
ratingsRDD = parts.map(lambda p: Row(userId=int(p[0]), movieId=int(p[1]),  
    rating=float(p[2]), timestamp=long(p[3])))
```

对数据文件的处理过程是，读取sample_movielens_ratings.txt，形成内存RDD；然后用“::”作为间隔符切割RDD的每一行；然后把每一行切割下来的第1个、第2个、第3个、第4个成分，分别解释为int、int、float、long等类型，分别赋予userId、movieId、rating、timestamp，构造一个Row对象，重新组织成一个新的RDD，即ratingsRDD

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例5: 协同过滤推荐
 - ◆ 训练集和测试集

```
#创建RDD, 并且进行训练集和测试集的划分。  
ratings = spark.createDataFrame(ratingsRDD)  
(training, test) = ratings.randomSplit([0.8, 0.2])
```

Spark MLlib

- ◆ Spark MLlib简介

- ◆ 实例5: 协同过滤推荐

- ◆ 创建ALS模型, 并进行训练。ALS模型的原理简述如下

- ◆ 交替最小二乘法(ALS)是统计分析中常用的一种逼近计算的算法。ALS的基础是最小二乘法(LS算法), 这是一种常用的机器学习算法, 通过最小化误差的平方和, 寻找数据的最佳函数匹配

	Rating Matrix(N*M)		
	Movie1	Movie2	Movie3
User1	5	3	5
User2	4	2	1
User3	0	3	3

M

	User Feature Matrix(F*N)		
	User1	User2	User3
F1	1	-4	1
F2	-2	0	-3
F3	0	-5	1

U

	Movie Feature Matrix(F*M)		
	Movie1	Movie2	Movie3
F1	-1	0	-2
F2	4	-4	1
F3	0	2	2

V

交替最小二乘法(ALS)原理

Spark MLlib

◆ Spark MLlib简介

◆ 实例5: 协同过滤推荐

◆ 假设现有用户对电影的评分矩阵M，里面有很多的空缺项，是个稀疏矩阵。

◆ 可以把这个评分矩阵，分解成两个矩阵，两个矩阵都不是稀疏矩阵，即U和V，U是用户的特征矩阵，V是电影的特征矩阵， $M = U^T V$

◆ 利用ALS进行求解的过程中，先固定矩阵U，求取V，然后再固定矩阵V，求取U矩阵；这样交替迭代计算，直到误差达到一定的阈值，或者达到一定的迭代次数为止。

◆ 求得U和V之后，我们就可以进行推荐。

◆ (1) 基于用户的推荐：利用用户特征矩阵U，就可以计算用户相似度。给用户u进行推荐的时候，就可以利用它的K个近邻。K个近邻评价比较高(看过觉得好)的电影，可以给用户u做推荐。

◆ (2) 基于物品的推荐：假设用户u看过电影m1和m2，评分比较高。我们就可以根据电影特征矩阵，找出和m1、m2相似的电影。利用用户对电影m1和m2的评分作为权重，适当对这些电影进行排序，推荐给用户u

$$\begin{pmatrix} 5 & 3 & 5 \\ 4 & 2 & 1 \\ 0 & 3 & 3 \end{pmatrix} = \begin{pmatrix} 1 & -4 & 1 \\ -2 & 0 & -3 \\ 0 & -5 & 1 \end{pmatrix}^T \begin{pmatrix} -1 & 0 & -2 \\ 4 & -4 & 1 \\ 0 & 2 & 2 \end{pmatrix}$$

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例5: 协同过滤推荐
 - ◆ ALS model
 - ◆ 可以把ALS模型的coldStartStrategy设置为drop, 以便保证不会得到NaN的分值(Evaluation Metrics), 即把代码改为`als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId", ratingCol="rating", coldStartStrategy="drop")`

```
# Build the recommendation model using ALS on the training data
als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId",
ratingCol="rating")
model = als.fit(training)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例5: 协同过滤推荐
 - ◆ 预测

```
#用测试集进行预测，评价指标是RMSE。  
# Evaluate the model by computing the RMSE on the test data  
predictions = model.transform(test)  
evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating",  
                                predictionCol="prediction")  
rmse = evaluator.evaluate(predictions)  
print("Root-mean-square error = " + str(rmse))
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例5: 协同过滤推荐
 - ◆ 预测

#如下代码显示30部Id为28的用户预测评分高的电影。

```
# predict
```

```
predictions = model.transform(test)
```

```
# select some columns
```

```
sub_predict = predictions.select("userId", "movieId", "rating", "prediction")
```

```
sub_predict.show(30)
```

```
# select userId = 28
```

```
sub_predict = sub_predict[ sub_predict['userId'].isin([28])]
```

```
sub_predict.show(30)
```

```
# sort asc
```

```
sub_predict = sub_predict.orderBy("prediction")
```

```
sub_predict.show(30)
```

```
# sort desc
```

```
from pyspark.sql.functions import desc
```

```
sub_predict = sub_predict.orderBy(desc("prediction"))
```

```
sub_predict.show(30)
```

```
#停止SparkSession。
```

```
spark.stop()
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例5: 协同过滤推荐
 - ◆ 预测

```
# 如下的代码，为每个用户各推荐10部电影，并且为每部电影各推荐10个用户。  
# 备注： Spark2.2.0 以上版本才支持如下接口。  
# Generate top 10 movie recommendations for each user  
userRecs = model.recommendForAllUsers(10)  
  
# Generate top 10 user recommendations for each movie  
movieRecs = model.recommendForAllItems(10)
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 实例5: 协同过滤推荐
 - ◆ 预测

如下的代码, 为特定用户各推荐10部电影, 并且为特定电影各推荐10个用户。

备注: Spark2.2.0以上版本才支持如下接口。

Generate top 10 movie recommendations for a specified set of users

```
users = ratings.select(als.getUserCol()).distinct().limit(3)
```

```
userSubsetRecs = model.recommendForUserSubset(users, 10)
```

Generate top 10 user recommendations for a specified set of movies

```
movies = ratings.select(als.getItemCol()).distinct().limit(3)
```

```
movieSubSetRecs = model.recommendForItemSubset(movies, 10)
```

pause



Spark MLlib

- ◆ Spark MLlib简介

- ◆ 为了运行本章介绍的机器学习实例，需要把相关的大数据平台软件启动起来。
- ◆ 具体来讲，需要启动HDFS、YARN、Hive metastore、以及Spark

```
rm -rf /opt/linuxsir/hadoop/logs/*.*
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*

clear
cd /opt/linuxsir/hadoop/sbin
./start-dfs.sh
./start-yarn.sh

clear
jps
ssh root@192.168.31.130 jps
ssh root@192.168.31.131 jps
```

Spark MLlib

- ◆ Spark MLlib简介

- ◆ 为了运行本章介绍的机器学习实例，需要把相关的大数据平台软件启动起来。
- ◆ 具体来讲，需要启动HDFS、YARN、Hive metastore、以及Spark

```
cd /opt/linuxsir/hive  
./bin/hive --service metastore -p 19083 &
```

\\ 等待metastore启动

```
netstat -lanp | grep 19083 \\确认metastore已经在19083端口监听
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 为了运行本章介绍的机器学习实例，需要把相关的大数据平台软件启动起来。
 - ◆ 具体来讲，需要启动HDFS、YARN、Hive metastore、以及Spark

```
cd /opt/linuxsir/spark  
./sbin/start-all.sh
```

```
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 启动完成后，需要运行pyspark
 - ◆ 以便运行分类、聚类、线性回归和协同过滤等机器学习实例，这些实例用Python语言来编写

```
cd /opt/linuxsir/spark
MASTER=spark://hd-master:17077 ./bin/pyspark

\\退出用quit()
```

Spark MLlib

- ◆ Spark MLlib简介
 - ◆ 完成机器学习实例的实验后，按照相反顺序，停止上述软件

```
cd /opt/linuxsir/spark  
./sbin/stop-all.sh
```

```
netstat -lanp | grep 19083  
kill -9 16336 \\16336是metastore进程号
```

```
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

```
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

pause

